Install Runbook — Windows + Arch dual-boot

Print this. Keep it next to the laptop.

Total wall-clock: ~90 min active, ~2 h with waits.

You'll need:

- Ventoy USB (already built + staged via `pnpm i` on the dev machine — whatever drive letter Windows mounts it at, e.g. `E:` or `F:`)
- Wi-Fi SSID + password
- The Bitwarden master password (for later — Bitwarden desktop in Arch)
- A second device (phone) to read this if you can't print
- A safe place to stash **two** recovery secrets: the **BitLocker recovery key** (see Phase 1 step 7) and the **LUKS passphrase** you pick in Phase 2d

What gets touched:

- **Samsung 512 GB** → EFI + MSR + Windows 160 GB (BitLocker) + Arch LUKS+btrfs (~316 GB)
- **Netac 128 GB** → Arch recovery ISO (unencrypted) + LUKS swap (16 GB, random key per boot) + LUKS ext4 for `/var/log` + `/var/cache`

Both drives are required. `install.sh` size-detects both and aborts if either is missing. If you've repurposed the Netac for something else, swap it back in before starting — the layout is not optional (swap, recovery partition, and the `/var/log` + `/var/cache` ext4 all depend on it per docs/decisions.md §Q9).

Three things to know before starting:

- **Hostnames intentionally match**: Windows = `Metis`, Arch = `metis`. Same name, same machine — your router sees `metis` regardless of which OS is up. (Windows may be renamed to `metis-win` later for unambiguous DHCP leases; not wired yet.)
- **First Windows boot after Arch install will prompt for the BitLocker recovery key.** Expected — installing **limine** to the shared EFI changes PCR values that BitLocker sealed against. Phase 1 step 7 stashes the recovery key in Bitwarden. See §C below for the one-shot reseal.
- **First Arch boot after install will prompt for the LUKS passphrase.** Also expected — TPM2 autounlock isn't wired until Phase 3 runs `systemd-cryptenroll`. After that, Arch boots silently (same model as BitLocker). The passphrase stays as a key slot forever so you can always recover.

Phase 0 — BIOS prep (5 min)

1. Plug the Ventoy USB. Power on. Hammer **F2** to enter BIOS setup.
2. Set:
   - **Boot Mode**: UEFI (not Legacy)
   - **Secure Boot**: **Disabled** for the install (limine UEFI binary isn't signed out of the box). `decisions.md` wants Secure Boot ON long-term; that's a separate later step using `sbctl` to enroll your own keys (see `phase-3-handoff.md` Upgrade Paths). Don't block on it now — get the install running first.
   - **SATA Operation**: AHCI (should already be; RAID/Intel RST will hide disks from Arch)

- **Fast Boot**: Disabled (stops it skipping the boot menu)
- **Fingerprint Reader**: **Enabled** (Dell BIOS hides this under Security → Fingerprint Reader on some firmware revisions). If it's disabled, Linux won't see the sensor at all and Phase 3 fingerprint enrollment will fail with "no devices".

3. Save & exit. Hammer **F12** at the Dell logo → boot menu → pick the USB (something like "UEFI: SanDisk ..." or the WD label).

If the USB isn't in F12: go back to BIOS → Boot Sequence → Add Boot Option, or make sure USB Boot is enabled. Secure Boot being on will also hide it.

Phase 1 — Windows install (~30 min, mostly unattended)

1. Ventoy menu appears. Pick `Win11_25H2_English_x64_v2.iso`.
2. If Ventoy shows "Boot in normal mode" / "Boot in grub2 mode" — pick **normal**.
3. `autounattend.xml` takes over. You'll see:
   - diskpart scrolls, Samsung gets wiped and laid out
   - Windows files copy (~10 min)
   - Reboots itself **twice** — leave the USB in both times; Ventoy will re-show the menu. **Don't press anything** — it'll auto-select the Win11 ISO via the `autosel` timer. If the timer expires and you're stuck, just re-pick Win11.
   - Finally lands on the `Tom` desktop. No prompts, no OOBE.

If you see `No unique 500-600 GB disk found - refusing to proceed`: The inline PowerShell safety check in `autounattend.xml` aborted because either zero or multiple disks fell in the 500–600 GB window. Likely causes and fixes, in order of likelihood:

- **An external drive is plugged in that's also 500–600 GB** → unplug it, press any key to dismiss the pause, power-cycle (hold power), retry F12 → USB.
- **The Samsung is missing from BIOS** → reboot into BIOS (F2). Confirm the Samsung SSD 840 PRO appears under Storage. If it doesn't: reseat the drive, or the SATA controller is still in RAID mode (flip to AHCI — Phase 0 step 2).
- **The Samsung's size is outside the window** → If you're running on different hardware than decisions.md §Q9 describes, the hardcoded 500–600 GB bounds are wrong. Fix: edit `autounattend.xml` — search for `500GB` and `600GB`, widen the window, re-stage the USB with `pnpm stage`.

4. **Wi-Fi should already be connected** — `autounattend.xml` embeds three profiles (`ATTgs5BwGZ`, `rhombus`, `rhombus_legacy`), all WPA2PSK, `connectionMode=auto`. Windows adds all three and connects to whichever is in range. Confirm via the taskbar network icon. If none are visible (different location), click the icon → pick SSID → enter password. The winget-import scheduled task waits up to 20 min for `cdn.winget.microsoft.com` and proceeds as soon as you're online.

5. Watch for winget to run:

    - It's a scheduled task called **`WingetImportOnce`**, starts ~2 min after logon.

- Open Task Manager → Details, you'll see `winget.exe` churning. Takes 10–20 min depending on your connection.
- It self-deletes the task on success.

If winget never runs or fails:

- Open PowerShell, run `schtasks /Query /TN WingetImportOnce /V /FO LIST` — look at **Last Result** (0 = ok, anything else = failed).
- Log files at `C:\Windows\Setup\Scripts\winget-*.log`.
- Manual fallback: `winget import -i C:\Windows\Setup\Scripts\winget-import.json --accept-package-agreements --accept-source-agreements --ignore-versions`

6. **BitLocker runs in parallel with winget** — it's in `FirstLogon.ps1`. With UsedSpaceOnly on a ~10-GB-used fresh install, encryption finishes in ~5 min. Check progress: `manage-bde -status C:` in PowerShell.
7. **Stash the BitLocker recovery key RIGHT NOW** — before doing anything else:
   - File #1: `C:\Windows\Setup\Scripts\BitLocker-Recovery.txt`
   - File #2: `<VENTOY_USB>:\bitlocker-recovery.txt` (same content)
   - Open it. Copy the 48-digit key.
   - Paste into Bitwarden as a **Secure Note** titled "Metis BitLocker recovery". Save.
   - Also take a phone photo of it as a belt-and-suspenders backup.
   - Once you've confirmed the key is safely in Bitwarden, delete both files: `Remove-Item C:\Window`

`s\Setup\Scripts\BitLocker-Recovery.txt, <USB_LETTER>:\bitlocker-recovery.txt -Force` (substitute whatever drive letter Windows assigned the Ventoy stick — check **This PC**)
    - **Do not skip this.** You'll need this key in Phase 2e — the first Windows boot after Arch install will almost certainly prompt for it.

8. Once winget is done (or even while it's running — Arch install doesn't care about Windows state), shut down. Leave the USB in.

Phase 2 — Arch install (~40 min)

2a. Boot the Arch live environment

1. Power on, F12, pick USB again.
2. Ventoy menu → pick **`archlinux-x86_64.iso`** → "Boot in normal mode". (The filename is always the undated symlink — `fetch-assets.ps1` mirrors that, not a dated release.)
3. At the Arch ISO menu, pick **"Arch Linux install medium (x86_64, UEFI)"**. You land at a root shell.

If step 3 fails with `ERROR: Failed to mount '/dev/loop0'` + `EXT4-fs: VFS: Can't find ext4 filesystem`: archiso copied the rootfs image to RAM but couldn't loop-mount it. Two causes, try in order:

1. **Ventoy ISO-virtualization hiccup** (tried first because it's a 30-second retry). Reboot, Ventoy menu, arrow onto `archlinux-x86_64.iso`, press **`d`** to toggle Memdisk mode (status indicator appears next to the filename), then Enter. Memdisk loads the whole ISO

into RAM before boot, bypassing Ventoy's loopback layer. The 7786's 16 GB RAM easily absorbs the 1.3 GB ISO.

2. **Corrupt ISO on the USB.** If Memdisk mode fails with the same error, the ISO file itself is bad (download or robocopy corrupted it — `fetch-assets.ps1` and `stage-usb.ps1` both SHA256-verify post-op now, but older sticks may predate that). Go back to the dev machine:

   ```
   # Replace V: with the Ventoy data drive letter
   CertUtil -hashfile V:\archlinux-x86_64.iso SHA256
   Get-Content V:\archlinux-sha256sums.txt | Select-St
   ring archlinux-x86_64.iso
   ```

   If the two hashes don't match, re-download + re-stage:

   ```
   pnpm restore:force   # re-downloads + verifies the
   Arch ISO
   pnpm stage:force     # re-copies + verifies on the
   USB
   ```

2b. Network

- **Ethernet via USB-C dock is the primary path** — plug it in before booting. `install.sh` pings `archlinux.org`; if ethernet is up, it skips Wi-Fi entirely.
- **Wi-Fi fallback**: if no ethernet, `install.sh` auto-connects using the embedded profiles (`ATTgs5BwGZ`, `rhombus`, `rhombus_legacy`). You do nothing.

If BOTH fail (dock ethernet driver missing from live ISO, all Wi-Fi networks out of range), fall back to manual iwctl:

```
iwctl
device list                                # note your
wifi device, probably wlan0
station wlan0 scan
station wlan0 get-networks
station wlan0 connect YOUR_SSID            # prompts fo
r passphrase
exit
ping -c2 archlinux.org
```

If `device list` is empty: `rfkill unblock all`, try again. If still empty, plug in the USB-C dock for wired ethernet.

2c. Fetch the installer

You can't just `mount /dev/disk/by-label/Ventoy` — it fails with "Can't open blockdev" because Ventoy holds the USB disk exclusively via dm-linear to serve the booted ISO. The install script works around that internally (creates a dm-linear passthrough and mounts the data partition at `/run/ventoy`, per ventoy.net/en/doc_compatible_mount.html), but you still need a way to invoke it. Pull a fresh copy from GitHub:

```
git clone https://github.com/fnrhombus/arch-setup /tm
p/arch-setup
```

2d. Run the installer

Optional but worth the 20 seconds if your connection feels slow: the live ISO ships with a stale/geographically-random mirrorlist. pacstrap on a bad mirror can appear to hang at <50 KB/s for 20+ min before it finishes. Refresh first:

```
reflector --latest 10 --sort rate --protocol https --
save /etc/pacman.d/mirrorlist
```

If `reflector` complains about no network, you haven't connected yet — go back to 2b. If it exits OK but the result still looks wrong (`head /etc/pacman.d/mirrorlist`), skip it — pacstrap will still work on the default list, just slower.

```
bash /tmp/arch-setup/phase-2-arch-install/install.sh
```

The script self-mounts the Ventoy data partition at `/run/ventoy` (for `chroot.sh` and the Arch ISO), so the clone is only needed to bootstrap this one invocation.

It will:

1. Size-detect the Samsung (500–600 GB) and Netac (100–150 GB). **Aborts loudly if either is missing** — don't blindly retry, fix it.
2. Show the plan and ask `[yes/NO]`. Type **`yes`** exactly.
3. **Prompt once up-front for three secrets** (all confirmed twice; nothing else will ask you for input until install is done):
   - **root password** — account recovery.
   - **tom's password** — your daily login. Make it strong; you'll bypass it with TPM-PIN and fingerprint later.
   - **LUKS passphrase** — unlocks both the Samsung root and Netac /var at boot until TPM2 autounlock is wired in Phase 3. **Stash this in Bitwarden now** (next to the BitLocker key) as a Secure Note called "Metis LUKS passphrase". 8+ chars, 12+ recommended. If PCR values ever drift, this is what gets you back in.
4. LUKS-format both data partitions, mkfs, pacstrap (~15 min — biggest wait, fully unattended).

5. Enter chroot (passwords + LUKS UUIDs handed in via mode-600 files), allocate TPM2 SHA-256 PCR bank, install **limine** + greetd + greetd-regreet, write `/etc/crypttab.initramfs` (cryptroot + cryptswap with TPM2) + `/etc/crypttab` (cryptvar via keyfile), add `sd-encrypt` to mkinitcpio HOOKS, install greetd config from `/root/arch-setup/phase-3-arch-postinstall/system-files/`, install pacman post-upgrade hook for TPM2 reseal on kernel/limine updates, wire greetd PAM for gnome-keyring auto-unlock.
6. `dd` the Arch ISO onto the Netac recovery partition (~1 min, unencrypted by design so it still boots from F12 if Arch is hosed).
7. Copy `postinstall.sh` + dotfiles into `/home/tom/`.
8. Unmount, close LUKS mappers, and print "Done."

If install.sh dies:

- **"No 500–600 GB disk detected"** → phase-1 Windows install didn't run. Boot back into Windows first.
- **"Samsung has <3 partitions"** → same — Windows didn't lay out the disk.
- **pacstrap fails** → network dropped. Reconnect (`iwctl` for Wi-Fi, or replug the dock for ethernet), then re-run. The script is **not idempotent**, so clear the partial state first:

```
umount -R /mnt
swapoff -a
# Wipe every partition that install.sh created on S
amsung + Netac.
# (lsblk shows them; the Samsung btrfs is the bigge
st partition on the 500–600 GB disk.)
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT /dev/sda /dev/sd
b 2>/dev/null
# For each Arch-owned partition (btrfs on Samsung,
swap + ext4 + ISO on Netac), wipefs:
# e.g. wipefs -a /dev/sda4 /dev/sdb1 /dev/sdb2 /de
v/sdb3
```

Then re-run `install.sh`. Recovery is ugly because it's a fresh install — if uncertain, just start over, it's fast.

- **"Could not find EFI System partition"** → Windows install put the EFI somewhere weird. Use `gdisk -l /dev/sda` to confirm GUID `c12a7328…`.
- **chroot script fails mid-way** → you can `arch-chroot /mnt` manually and finish the remaining steps from `chroot.sh` by hand.

9. When it says "Done": `reboot`. Pull the USB as it powers off.

2e. First boot into Arch

- limine boot menu appears with:
  - **Arch Linux** (linux kernel) + **Arch Linux (fallback)**
  - **Arch Linux LTS** (linux-lts kernel) + **Arch Linux LTS (fallback)** — the safety net if a linux upgrade breaks something
  - Auto-discovered **Windows Boot Manager**

  3-sec timeout; Arch Linux (the non-LTS) is default.

- **LUKS passphrase prompt** appears early in boot: `Please enter passphrase for disk (cryptroot):`. Type

the passphrase you set in Phase 2d. The /var volume auto-unlocks from a keyfile — only cryptroot asks you for anything. After Phase 3's `systemd-cryptenroll` runs, this prompt is replaced by silent TPM unlock.

- It boots to a black screen, then to **greetd + ReGreet** (graphical login).

If the limine menu doesn't show Windows: boot into Arch anyway, then as root check `/boot/limine.conf` — the `Windows Boot Manager` chainload entry should be present (chroot.sh writes it). Verify `ls /boot/EFI/Microsoft/Boot/bootmgfw.efi` exists — missing file means Windows install didn't write it (rare). Fix later; don't block on this.

Expected: BitLocker recovery prompt on first Windows boot after this step. limine installing itself to the shared EFI changes the TPM's PCR values → BitLocker can't auto-unseal → you get the blue "Enter recovery key" screen. This is not a bug.

- Type the 48-digit key from Bitwarden.
- Windows unlocks, boots normally.
- Windows re-seals to the new PCR values automatically. Next boots: silent unlock, no prompt ever again (unless bootloader changes).
- **If you don't have the key when this screen appears — you are locked out of Windows.** Phase 1 step 7 exists for this reason.

If nothing boots: BIOS → Boot Sequence → make sure "Limine Boot Manager" is listed and first. If only "Windows Boot Manager" is there, recover from a live USB: unlock LUKS + chroot in (see §A below for the unlock

incantation), then `install -d /boot/EFI/BOOT && install -m 644 /usr/share/limine/BOOTX64.EFI /boot/EFI/BOOT/BOOTX64.EFI && efibootmgr --create --disk /dev/sda --part 1 --label "Limine Boot Manager" --loader '\EFI\BOOT\BOOTX64.EFI'` to re-register.

2f. Clear the BitLocker prompt now (before Phase 3)

Do this before starting Phase 3 — you want the BitLocker key mess handled while you're still thinking about it, not 25 minutes into Hyprland setup.

1. Reboot. At the limine boot menu, pick **Windows Boot Manager**.
2. BitLocker prompts for the recovery key. Type the 48-digit key from Bitwarden ("Metis BitLocker recovery").
3. Windows boots. Log in as `Tom`. Let it sit for 30 seconds so BitLocker re-seals against the new PCR values.
4. Shut down: **Start → Power → Shut down**. (Fast Startup is off, so this is a clean shutdown, not a hybrid-hibernate.)
5. Power on → limine → pick **Arch Linux** → back into Arch. No more BitLocker prompts from here.

If the BitLocker key doesn't work: you have the phone photo from Phase 1 step 7 as a last-ditch copy. If even that doesn't work, Windows is gone and you need to reinstall — Arch is unaffected.

Phase 3 — Post-install (~25 min)

3a. Get a terminal

At the **greetd + ReGreet** login screen, log in as **tom**. Hyprland starts but there's no config applied yet (chezmoi hasn't run) — expect a blank screen, no keybindings, no launcher.

Switch to a TTY: `Ctrl + Alt + F2` → text login as `tom`.

3b. Run postinstall

```
chmod +x ~/postinstall.sh
~/postinstall.sh
```

It will:

1. `sudo` prompt for your password — type it. (No fingerprint/PIN yet.)
2. **pacman** — base CLI tooling + the bare-Hyprland stack (waybar, swaync, fuzzel, swayosd, hyprlock, hypridle, hyprpolkitagent, hyprpicker, nm-connection-editor, pwvucontrol, nwg-displays, qt5/6ct, papirus-icon-theme, imv, zathura, etc.) + Bitwarden + Ghostty + cliphist + satty + hyprshot + mise + chezmoi + gh + snapper. Signed binaries from `extra`, ~5–8 min.
3. **yay** — AUR packages: visual-studio-code-bin, microsoft-edge-stable-bin, claude-desktop-native, pinpam-git, sesh, iio-hyprland-git, powershell-bin, **awww** (Wayland wallpaper), **matugen-bin** (Material You), mission-center, overskride, wleave, bibata-cursor + bibata-cursor-translated, pacseek. ~10 min build time.

4. Installs Claude Code CLI: `mise use -g node@lts` pulls a LTS Node, `npm install -g @anthropic-ai/claude-code` bootstraps, then `claude install` migrates to the **native install** at `~/.claude/local/` so auto-updates don't need sudo. **No local SSH keys are generated** — keys live in your Bitwarden vault as "SSH key" items and surface via `~/.bitwarden-ssh-agent.sock` once Bitwarden desktop is running with the SSH-agent toggle on (Phase 3e).
5. **Points Bitwarden at your self-hosted server** `https://hass4150.duckdns.org:7277` — CLI via `bw config server`, desktop via pre-seeded `~/.config/Bitwarden/data.json`.
6. **Prompt you to enroll your fingerprint** — touch the sensor 5 times. Reader is auto-detected (Goodix 538C via `libfprint-goodix-53xc`).
7. **Prompt you to set a TPM-PIN** (`pinutil setup`) — 6+ chars. PAM module name is `libpinpam.so` (NOT `pam_pinpam.so` — quirk of the AUR package). 7.5. **Enroll TPM2 on BOTH ArchRoot AND ArchSwap** (`systemd-cryptenroll --tpm2-pcrs=0+7`). Both volumes get silent unseal at next boot. Hibernate-ready cryptswap requires TPM2 enrollment so `resume=` works without the LUKS passphrase prompt.
8. Wires `~/.ssh/config` for Bitwarden SSH agent.
9. Plants first-login + ssh-signing scripts in `~/.zshrc.d/`.
0. Builds zgenom plugins (warms cache so first login is fast).
1. Takes a **snapper baseline snapshot** of `/`.
2. Installs USB-serial udev rules (ESP32/Pico/FTDI/CH340) and adds you to `uucp`.

3. **Runs `chezmoi init --source=/root/arch-setup/dotfiles && chezmoi apply`** — writes Hyprland configs (entry + 9 fragments), waybar, swaync, fuzzel, ghostty, yazi, helix, imv, zathura, qt5ct/qt6ct, matugen pipeline + templates, and the helper scripts (theme-toggle, wallpaper-rotate, control-panel, validate-hypr-binds). The `run_once` wallpaper bootstrap downloads from `fn rhombus/callisto` into `~/Pictures/Wallpapers/`.
4. **Loads Hyprland plugins via `hyprpm`**: hyprexpo (workspace overview) + hyprgrass (touch gestures). Note: must re-run from inside Hyprland to actually take effect — re-runs are idempotent.
5. Prints a verify table — scan for **FAIL** rows.

If fingerprint enroll fails: postinstall already handles the Goodix fallback interactively. If you declined or it still fails:

```
SKIP_FPRINT=1 ~/postinstall.sh    # re-run skipping f
inger step
```

Then manually: `lsusb | grep -iE 'goodix|validity|synaptics'`, `fprintd-list-devices`. Check https://fprint.freedesktop.org/supported-devices.html for your VID:PID.

If `pinutil` is missing after AUR install:

```
SKIP_PIN=1 ~/postinstall.sh       # re-run skipping P
IN step
# Then manually:
yay -S pinpam-git
sudo pinutil setup
```

If something else fails mid-run: re-run `~/postinstall.sh`. It's idempotent.

3c. Reboot into Hyprland

```
sudo reboot
```

- greetd + ReGreet again. Log in as tom.
- Hyprland comes up with the chezmoi-applied configs: waybar at top, matugen-themed wallpaper, ~85 keybindings live.
- Default terminal: `Super + Return` opens Ghostty.
- App launcher: `Super + Space` (fuzzel).
- Cheat sheet: `runbook/keybinds.md` (printable).
- The first matugen render runs from Hyprland's `exec-once = ~/.local/bin/wallpaper-rotate --first` — wallpaper appears, palette derives, all components reload.

Escape hatch if NO keybinding opens a terminal (keybind variant mismatch, Hyprland config didn't install, wayland handshake fail): `Ctrl + Alt + F3` drops to TTY3 where you can log in as `tom` and debug. `Ctrl + Alt + F1` jumps back to the greetd/Hyprland session. Worst case: from TTY3, `journalctl --user -u hyprland -b` tells you why Hyprland dropped you to a featureless compositor.

3e. Bitwarden one-time setup

1. Launch Bitwarden from the launcher.
2. Log in with your master password (only time you type it).
3. **Settings → Security → enable "Unlock with system keyring".** You'll be asked for the master password once more — after that, gnome-keyring holds it and Bitwarden auto-unlocks at login.

4. **Settings → SSH agent → Enable**. The socket appears at `~/.bitwarden-ssh-agent.sock` (already wired into `~/.ssh/config`).
5. Add any SSH keys you want as **"SSH key"** vault items.
6. **You MUST log out and back in now** (or at least close + reopen every terminal). `SSH_AUTH_SOCK` is set by `~/.zshrc.d/bitwarden-ssh-agent.zsh` at shell start — it only notices the new socket in a fresh shell. Skip this and the 3f sanity check `ssh-add -l` will fail even though the agent is actually working.

After the re-login: Bitwarden auto-unlocks (via gnome-keyring), and `ssh-add -l` lists your keys with no prompt.

3e-bis. Azure DDNS one-time setup (`metis.rhombus.rocks`)

`postinstall.sh` installs the `metis-ddns` script + systemd timer + NetworkManager dispatcher hook + the `az` CLI, and stubs `/etc/metis-ddns.env` from a template. **You still have to wire up the service-principal credentials once** — but that's automated by `setup-azure-ddns.sh` (also copied to `~/` by install.sh).

The script is fully idempotent: each run reuses an existing `metis-ddns` app registration if found, ensures the SP exists, **rotates the secret** (replaces, doesn't append), ensures the role assignment, and rewrites both `/etc/metis-ddns.env` and `/etc/letsencrypt/azure.ini` with fresh creds.

It uses piecemeal `az ad app` / `az ad sp` / `az role assignment` commands instead of the all-in-one `az ad sp create-for-rbac` because the latter is broken under Python

3.14 + azure-cli 2.85.0 (argparse `%Y` bug, observed 2026-04-23).

```
sudo -v                                   # cache sudo
creds for the script's installs
az login                                  # device-cod
e flow, opens browser
~/setup-azure-ddns.sh                     # ~5 sec wal
l clock; restarts metis-ddns at the end
sudo journalctl -u metis-ddns -n 30       # confirm "s
tatus=0/SUCCESS"
```

First service call may 403 with "AuthorizationFailed" — Azure role assignments propagate in 30s–5min. Wait, retry. After first success, the timer + NM hook take over and you don't think about it again until the SP secret expires in 2 years (the script will rotate when re-run).

Verify from the outside (any machine, even your phone):

```
dig AAAA metis.rhombus.rocks +short
```

Should return your laptop's current public IPv6 address.

3e-ter. Let's Encrypt cert for `metis.rhombus.rocks`

Only useful **after step 3e-bis succeeds** (DNS must resolve before the dns-01 challenge will).

`postinstall.sh` installs `certbot` + the `certbot-dns-azure` plugin (via pipx — not packaged for Arch). `setup-azure-ddns.sh` already wrote the SP creds into `/etc/letsencrypt/azure.ini` using the plugin's own INI keys, so there's nothing to fill in by hand.

Issue the cert (one-time):

```
sudo certbot certonly \
    --authenticator dns-azure \
    --dns-azure-credentials /etc/letsencrypt/azure.in
i \
    --dns-azure-propagation-seconds 60 \
    -d metis.rhombus.rocks \
    --agree-tos -m <your-email> --no-eff-email
```

Cert lands at `/etc/letsencrypt/live/metis.rhombus.rocks/{fullchain,privkey}.pem`. `certbot-renew.timer` is already enabled by postinstall — it'll renew within 30d of expiry, twice daily, with no further action.

3e-quater. Firewall — confirm and tweak

`postinstall.sh` enables `ufw` with default-deny incoming, default-allow outgoing, and `22/tcp ALLOW` for SSH (Callisto pubkey already in `~/.ssh/authorized_keys`). Rules apply to **both IPv4 and IPv6** — ufw is dual-stack out of the box.

```
sudo ufw status verbose                  # confirm "S
tatus: active", "Default: deny (incoming)"
sudo ufw allow <port>/tcp                # add a rule
sudo ufw delete allow <port>/tcp         # remove a r
ule
```

Why this matters: your router can globally enable/disable IPv6 TCP to Metis but can't filter per port. With IPv6 there's no NAT — every device gets a publicly routable address, so anything bound to a port on the laptop is exposed when the router-side toggle is on. ufw is the per-port gate on the host side.

3f. Sanity checks

```
# sudo: clear cache, then test. Pay attention to WHIC
H prompt appears.
sudo -k
sudo echo ok
```

The prompt you see tells you what's wired:

- `PIN:` → pinpam is working. TPM-PIN is your daily sudo auth. ✓
- `Place your finger on the fingerprint reader` → fprintd is working. ✓
- `[sudo] password for tom:` → **neither TPM-PIN nor fingerprint is wired**. Something went wrong. See recovery §I below before relying on this.

```
# fingerprint enrolled?
fprintd-list tom               # should show a finger
print enrolled

# PAM wiring
sudo grep -E 'pam_pinpam|pam_fprintd' /etc/pam.d/sudo
# Expect both lines, both marked `sufficient`, before
the pam_unix.so fallback.

# ssh agent
echo $SSH_AUTH_SOCK            # should be ~/.bitward
en-ssh-agent.sock (set by ~/.zshrc.d/bitwarden-ssh-ag
ent.zsh)
ssh-add -l                     # should list keys onc
e Bitwarden desktop is unlocked + SSH-agent toggle is
on
```

Phase 4 — Start the Claude session with the handoff

1. Open Ghostty. Start tmux: `tmux new -s main` (prefix = **Ctrl+a**).
2. Clone the setup repo so Claude has context:

   ```
   mkdir -p ~/src
   cd ~/src
   # Primary: clone from GitHub (if repo is pushed).
   git clone git@github.com:fnrhombus/arch-setup.git a
   rch-setup@fnrhombus
   cd arch-setup@fnrhombus
   ```

   If the clone fails (repo not pushed, or gh auth not wired yet), recover from the Ventoy USB — it still has the repo contents synced there pre-install:

```
# Re-insert the Ventoy USB (you pulled it at end of
Phase 2d).
sudo mkdir -p /mnt/ventoy
sudo mount /dev/disk/by-label/Ventoy /mnt/ventoy

# Sanity-check what's actually at the USB root befo
re copying — the sync
# step (pre-install) may have only pushed the phase
-2/3 script dirs and
# not the markdown docs. If the `.md` files are mis
sing, Claude still has
# enough to proceed with just phase-3 script + CLAU
DE.md.
ls /mnt/ventoy/*.md /mnt/ventoy/CLAUDE.md 2>/dev/nu
ll

mkdir -p ~/src/arch-setup@fnrhombus
# Copy every repo file that IS on the USB; missing
ones silently skip
# thanks to 2>/dev/null. Falls through to a whole-U
SB snapshot only if
# the targeted copy produced no files at all.
cp -r /mnt/ventoy/phase-1-windows \
      /mnt/ventoy/phase-2-arch-install \
      /mnt/ventoy/phase-3-arch-postinstall \
      /mnt/ventoy/runbook \
      /mnt/ventoy/docs \
      /mnt/ventoy/autounattend.xml \
      /mnt/ventoy/CLAUDE.md \
      /mnt/ventoy/phase-6-grow-windows.sh \
      ~/src/arch-setup@fnrhombus/ 2>/dev/null
if [[ -z "$(ls -A ~/src/arch-setup@fnrhombus 2>/de
v/null)" ]]; then
    warn "Targeted copy got nothing — falling back
to a whole-USB snapshot."
    cp -r /mnt/ventoy ~/src/arch-setup-snapshot
    cd ~/src/arch-setup-snapshot
else
    cd ~/src/arch-setup@fnrhombus
fi
```

3. Start Claude in that directory:

```
claude
```

4. Feed it the handoff as the first message:

```
Read runbook/phase-3-handoff.md. That's the brief.
Start by walking
me through the Hyprland keybindings — I'm looking a
t a fresh
bare-Hyprland session with Claude-authored configs
in chezmoi
(matugen theme) and I don't know how to do anything
yet.
```

   Claude will read `runbook/phase-3-handoff.md`, `docs/decisions.md`, `docs/desktop-requirements.md`, and `CLAUDE.md` on its own and have the full context. The keybind cheat sheet is at `runbook/keybinds.md`.

Things that will eat time but aren't broken

- **pacstrap and yay builds look stuck** — they're not. `base-devel` pulls ~50 packages, AUR builds compile from source. Don't Ctrl-C.
- **zgenom first-login rebuild** — if postinstall's pre-build didn't fully warm the cache, your first `zsh` login will take 20 seconds. One-time.
- **chezmoi apply** — first run downloads wallpapers from the callisto repo and writes ~50 dotfiles. ~30 sec.
- **fingerprint enroll** — takes 5 touches but the reader is picky about angle. If it keeps rejecting, lift fully off between touches.
- **First Hyprland start** — matugen renders the initial palette + all components reload. ~5 sec lag before the bar appears.

Things that are actually broken if you see them

- **"No network" during pacstrap** → wifi dropped. Reconnect, restart install.
- **limine boot menu missing entries** → check `/boot/limine.conf`; if the Linux entries are gone, reinstall the limine UEFI binary: `install -d /boot/EFI/BOOT && install -m 644 /usr/share/limine/BOOTX64.EFI /boot/EFI/BOOT/BOOTX64.EFI && efibootmgr --create --disk /dev/sda --part 1 --label "Limine Boot Manager" --loader '\EFI\BOOT\BOOTX64.EFI'`.
- **greetd loops back to login** → Hyprland is crashing. Drop to TTY (Ctrl+Alt+F3), `journalctl --user -u hyprland -b -n 100`. Usually a chezmoi-apply mismatch (config drift from upstream Hyprland). Re-run `chezmoi apply` and check `validate-hypr-binds` for keybind conflicts.
- **`sudo` rejects your TPM-PIN** → TPM got reset or the PIN index was evicted. Re-run `sudo pinutil setup`. Password still works as fallback.

First-boot failure modes (will it all actually come up?)

These are the failure modes most likely to hit on the very first boot after `install.sh` reboots. Each has a recovery path that assumes you have the Ventoy USB handy.

A. Kernel panic: "Cannot open root device" / "VFS: Unable to mount root fs"

Cause: btrfs module didn't make it into the initramfs, so the kernel boots but can't read `/`. `chroot.sh` now forces `MODULES=(btrfs)` in `/etc/mkinitcpio.conf` — but if that line got overridden or mkinitcpio threw a warning you ignored, this is what you see.

Fix: Boot the Ventoy USB → Arch ISO → live environment. Then:

```
# Unlock the LUKS container first — /dev/disk/by-labe
l/ArchRoot is the btrfs
# label inside LUKS, so it only appears after `crypts
etup open`. Use the
# raw partition via PARTLABEL (set by install.sh sgdi
sk).
cryptsetup open /dev/disk/by-partlabel/ArchRoot crypt
root
# Prompt for the LUKS passphrase (the one from Phase
2d / Bitwarden).
mount -o subvol=@ /dev/mapper/cryptroot /mnt
# Find the EFI partition by GPT type GUID — Windows d
iskpart doesn't
# assign a PARTLABEL, so by-partlabel lookups are unr
eliable.
EFI=$(lsblk -rno NAME,PARTTYPE | awk '$2=="c12a7328-f
81f-11d2-ba4b-00a0c93ec93b"{print "/dev/"$1; exit}')
mount "$EFI" /mnt/boot
arch-chroot /mnt
grep -q '^MODULES=(btrfs)' /etc/mkinitcpio.conf || se
d -i 's/^MODULES=.*/MODULES=(btrfs)/' /etc/mkinitcpi
o.conf
mkinitcpio -P
exit
umount -R /mnt
cryptsetup close cryptroot
reboot
```

B. Emergency shell at boot: "Dependency failed for /var/log"

Cause: The `/var/log` and `/var/cache` bind mounts race the ext4 mount at `/mnt/netac-var`. `install.sh` now writes `x-systemd.requires-mounts-for=/mnt/netac-var` into both fstab entries, but if the fstab-rewrite step silently skipped (look for `[!] fstab post-process failed` in install output), the ordering won't be there.

Fix (from the emergency shell at boot):

```
# Log in as root (password you set in chroot).
mount -o remount,rw /
vim /etc/fstab   # or `vi` — nano is NOT installed, only vim and helix are pacstrapped
# Find the two lines whose target is /var/log and /var/cache.
# In their options column (4th column), append:
#   ,x-systemd.requires-mounts-for=/mnt/netac-var
# Save with :wq
systemctl daemon-reload
reboot
```

If editing fstab from emergency mode is scary: add `systemd.unit=rescue.target` to the kernel command line in **limine** (press `e` on the Arch entry at boot, append to the `cmdline:` line), which gives a full single-user shell with `/` writable.

C. BitLocker recovery prompt on next Windows boot (and keeps prompting)

First prompt is expected, not a failure. Installing limine (or any non-Microsoft EFI binary) to the shared EFI rewrites the PCR values the TPM sealed BitLocker against. First Windows boot after Arch install → blue "Enter recovery key" screen. Type the 48-digit key (stored in

Bitwarden as "Metis BitLocker recovery" per Phase 1 step 7) to get in.

But Windows does NOT auto-reseal the TPM protector just because you entered the recovery key — that path unlocks the drive but leaves the TPM protector sealed against the *old* (pre-Arch) PCRs. So the next boot, and the next, and the next will all keep prompting unless you force a re-seal.

Fix (elevated PowerShell, after you've unlocked into Windows with the recovery key):

```
# 1. Sanity-check Fast Startup is OFF (we disable it
via autounattend, but confirm
#    — resume-from-hibernate shuffles PCRs and would
re-break the seal on every wake).
#    Fast Startup requires hibernation; if hibernatio
n is off, Fast Startup is off too.
powercfg /a
#    Look for "Hibernation has not been enabled" in t
he NOT-available section.
#    DO NOT grep for "hybrid" — that matches "Hybrid
Sleep", a different feature.
#    Definitive registry check (0 = off = good, 1 = o
n, missing = hiberation off):
(Get-ItemProperty 'HKLM:\SYSTEM\CurrentControlSet\Con
trol\Session Manager\Power' -Name HiberbootEnabled -E
rrorAction SilentlyContinue).HiberbootEnabled
# If Fast Startup is on, disable it first:
#   powercfg /h off

# 2. Suspend BitLocker until manually re-enabled. Dat
a stays encrypted; the unlock
#    key is stored in clear on disk temporarily. -Reb
ootCount 0 means "stay
#    suspended until I say so" (not "0 reboots remain
ing").
manage-bde -protectors -disable C: -RebootCount 0

# 3. Reboot. No BitLocker prompt this time (protectio
n suspended).
shutdown /r /t 0

# 4. Back in Windows, re-enable. This re-seals the TP
M protector against
#    *current* PCR values (i.e. the new limine → WBM
→ Windows chain).
manage-bde -protectors -enable C:

# 5. Confirm: TPM protector should show as "Enabled"
again.
manage-bde -protectors -get C:
```

After that, every subsequent boot unlocks via TPM silently. The boot chain needs to stay stable for this to

hold — if something later changes it (limine update rewriting its EFI binary, Windows update replacing the boot manager, firmware update changing Secure Boot state), you'll get prompted once more. The pacman post-upgrade hook `/etc/pacman.d/hooks/95-tpm2-reseal.hook` automates the re-seal on linux/limine/mkinitcpio updates so this is normally invisible.

Nuclear option: turn BitLocker off. You already have LUKS on the Linux side. If your Windows partition doesn't need encryption-at-rest (home dev laptop, not regulated-device territory), decrypting removes the entire class of problem:

```
manage-bde -off C:         # background decryption, ~
30-60 min for 160 GB
manage-bde -status C:      # watch progress
```

Trade-off: stolen laptop + removed drive + NTFS reader = your Windows files are readable.

D. DHCP lease shows two devices both named `metis`

Cause: Both OSes intentionally use the same hostname (`Metis` on Windows, `metis` on Arch — case-insensitive at the DHCP/mDNS layer). Your router will see one `metis` lease that flips between two MAC addresses depending on which OS is up.

This is by design for the single-OS-at-a-time use case. If you want unambiguous leases (e.g., for both OSes to coexist visibly in router DHCP tables, or to avoid mDNS collisions if both are ever booted simultaneously via PXE/dual-network):

- **Rename Windows**: `Rename-Computer -NewName metis-win -Restart` from elevated PowerShell.
- **Or rename Arch**: `sudo hostnamectl set-hostname metis-arch` (also update the `127.0.1.1` line in `/etc/hosts`).

Neither change is required for anything to work.

E. Snapper complains `config "root" does not exist` or `.snapshots not found`

Cause: `postinstall.sh` skips `snapper -c root create-config /` (which would try to re-create an already-mounted `/.snapshots` subvolume) and writes the config by hand. If that hand-write was skipped — e.g., `postinstall.sh` was re-run after partial state — you're missing the config.

Fix:

```
sudo install -d -m 750 /etc/snapper/configs
sudo cp /etc/snapper/config-templates/default /etc/snapper/configs/root
sudo sed -i 's|^SUBVOLUME=.*|SUBVOLUME="/"|;s|^ALLOW_USERS=.*|ALLOW_USERS="tom"|' /etc/snapper/configs/root
# Guard against double-append if you're re-running this:
grep -q '^SNAPPER_CONFIGS=.*root' /etc/conf.d/snapper 2>/dev/null || \
  echo 'SNAPPER_CONFIGS="root"' | sudo tee -a /etc/conf.d/snapper
sudo chown :tom /.snapshots 2>/dev/null || true
sudo chmod 750 /.snapshots
sudo snapper -c root create --description "manual baseline"
```

F. Bluetooth doesn't auto-enable after reboot

Cause: The AutoEnable sed rewriters in `chroot.sh` all silently no-op'd (a distro change in `/etc/bluetooth/main.conf` defaults would do it). There's now a final `>>` append fallback, but if the config file didn't exist at all when chroot.sh ran, the whole `if` block was skipped.

Fix:

```
sudo tee -a /etc/bluetooth/main.conf >/dev/null <<EOF

[Policy]
AutoEnable=true
EOF
sudo systemctl restart bluetooth
```

G. Can't get back into Windows from the Arch limine boot menu

Cause: the Windows Boot Manager chainload entry is missing from `/boot/limine.conf` (chroot.sh writes it explicitly under `/Windows Boot Manager`). Check the file; if absent, append:

```
/Windows Boot Manager
    protocol: efi_chainload
    image_path: boot():/EFI/Microsoft/Boot/bootmgfw.e
fi
```

Verify `ls /boot/EFI/Microsoft/Boot/bootmgfw.efi` exists — missing file means Windows install didn't write it.

Fix: Reboot, hit F12 at the Dell logo, pick "Windows Boot Manager" directly from the firmware menu. From a live USB later, boot the Arch ISO, unlock LUKS + chroot in (see §A above for the unlock incantation), and re-run `limine b`

`ios-install` + reinstall the UEFI binary to `/boot/EFI/BOOT/BOOTX64.EFI` to re-register. You are not stuck — both OSes are still bootable via F12.

H. greetd shows but fingerprint prompt never appears at login

The greetd PAM stack (`/etc/pam.d/greetd`) DOES include `pam_fprintd.so sufficient max-tries=1 timeout=10` — fingerprint is the first auth attempt at the greeter. If it doesn't appear:

- Confirm fprintd is enrolled: `fprintd-list tom`
- Check the greetd PAM stack actually has the line: `cat /etc/pam.d/greetd`
- Restart greetd: `sudo systemctl restart greetd`

Note: PIN is **deliberately excluded** from greetd (cold-boot wants full credential per Windows Hello pattern). Sudo + hyprlock include PIN per `postinstall.sh §7a`.

I. `sudo` fails with "PAM module not found" — you're locked out of root

Cause: If `pinpam-git` failed to install but `postinstall.sh`'s PAM wiring ran anyway (or vice versa), `/etc/pam.d/sudo` references a `pam_pinpam.so` module that doesn't exist on disk. Same for `pam_fprintd.so` if fprintd was uninstalled. PAM's error mode for a missing module is to **fail the entire auth stack**, not fall through — which means your password won't work either. You are hard-locked.

Fix — from the Ventoy USB Arch live environment:

```
# Boot Ventoy → Arch ISO → live env → connect wifi.
cryptsetup open /dev/disk/by-partlabel/ArchRoot crypt
root   # LUKS passphrase
mount -o subvol=@ /dev/mapper/cryptroot /mnt
arch-chroot /mnt

# Strip every non-standard auth line from /etc/pam.d/
sudo back to defaults.
cat > /etc/pam.d/sudo <<'EOF'
#%PAM-1.0
auth       include      system-auth
account    include      system-auth
session    include      system-auth
EOF
exit
umount -R /mnt; cryptsetup close cryptroot
reboot
```

After reboot, `sudo` takes your **password** (TPM-PIN + fingerprint are gone until you re-wire PAM). Fix the missing module: reinstall `pinpam-git` or `fprintd`, then re-run the PAM wiring blocks from `postinstall.sh` by hand.

Prevention: after `postinstall.sh` finishes, always test sudo from a **second** terminal before closing the one you ran the script in. If sudo fails, you can fix PAM from the still-privileged parent session without rebooting.

Sibling issue — locked out of greetd login: the same kind of break can hit `/etc/pam.d/greetd` (if fprintd is somehow uninstalled or the PAM module is renamed). Symptom: greetd rejects every password. Recovery is identical to the sudo case — boot the Ventoy USB or Netac recovery → Arch live → unlock LUKS → chroot → restore the file:

```
cryptsetup open /dev/disk/by-partlabel/ArchRoot crypt
root   # LUKS passphrase
mount -o subvol=@ /dev/mapper/cryptroot /mnt
arch-chroot /mnt
cat > /etc/pam.d/greetd <<'EOF'
#%PAM-1.0
auth       include      system-login
account    include      system-login
password   include      system-login
session    include      system-login
EOF
exit
umount -R /mnt; cryptsetup close cryptroot
reboot
```

Password login returns; re-wire fingerprint after you've reinstalled fprintd.

J. `ssh-add -l` returns "error fetching identities" in every new shell

Cause: Bitwarden desktop isn't running, OR Bitwarden's SSH-agent toggle is off, OR the socket moved. `~/.zshrc.d/bitwarden-ssh-agent.zsh` only exports `SSH_AUTH_SOCK` if the socket file exists, so if the socket is gone, `ssh-add` talks to no agent and errors.

Fix:

1. Check the socket exists: `ls -l ~/.bitwarden-ssh-agent.sock`
2. If missing: launch Bitwarden desktop, unlock, Settings → SSH agent → toggle on. Quit and reopen a terminal.
3. If present but `ssh-add -l` still errors: `SSH_AUTH_SOCK=~/.bitwarden-ssh-agent.sock ssh-add -l` to rule out an env issue.
4. If the SSH-signing planter at `~/.zshrc.d/arch-ssh-signing.zsh` was never deleted, it will try again on each

login — once Bitwarden is back online with at least one "SSH key" vault item, the planter fires and self-deletes.

K-pre. chezmoi apply failed in postinstall §13

Cause: `postinstall.sh §13` runs `chezmoi init --source=/root/arch-setup/dotfiles && chezmoi apply`. If the source dir wasn't staged at `/root/arch-setup/` (install.sh §11 should have copied it there), chezmoi has nothing to apply and Hyprland comes up with empty config.

Fix:

```
ls /root/arch-setup/dotfiles/dot_config/hypr/   # should exist
# If empty:
sudo cp -r /run/ventoy/dotfiles /root/arch-setup/
# if Ventoy still mounted
# Or:
sudo git clone https://github.com/fnrhombus/arch-setup.git /root/arch-setup
chezmoi init --source=/root/arch-setup/dotfiles
chezmoi apply --force
```

Re-login (or `Super+Shift+R` to reload Hyprland) once apply finishes.

K-binds. validate-hypr-binds reports duplicate (MOD, KEY) pairs

The validator script catches binding conflicts as a chezmoi pre-apply hook. If it blocks `chezmoi apply` with a `DUP` or `UNK` issue:

```
~/.local/bin/validate-hypr-binds       # see the conf
licts
hx ~/.config/hypr/binds.conf           # fix the dupl
icate or rename to a new dispatcher
~/.local/bin/validate-hypr-binds       # re-validate;
should report OK
chezmoi apply
```

After a clean validation, regenerate the printed cheat sheet:

```
cd /root/arch-setup
~/.local/bin/validate-hypr-binds --emit-cheatsheet
```

L-nvidia. NVIDIA modules load despite the blacklist (screen tearing, GBM errors)

Cause: `/etc/modprobe.d/blacklist-nvidia.conf` is written but the modules are already baked into the initramfs, OR a kernel upgrade pulled them back in.

Fix:

```
# Verify the blacklist file exists and has the expect
ed lines:
cat /etc/modprobe.d/blacklist-nvidia.conf
# Expected: blacklist nouveau / blacklist nvidia / bl
acklist nvidia_drm / etc.
# Rebuild the initramfs for all kernels so the blackl
ist takes effect at boot:
sudo mkinitcpio -P
# Confirm no NVIDIA modules are loaded after reboot:
lsmod | grep -iE 'nvidia|nouveau'     # expect empty
output
```

If modules still load: check `/etc/mkinitcpio.conf` — `MODULES=(btrfs)` should NOT contain `nvidia` or `nouveau`. Check `/etc/modules-load.d/` for any file forcing them.

M-luks. LUKS prompt rejects your passphrase at boot

Cause: Most likely you fat-fingered it during Phase 2d (same failure mode as the account passwords below — the `prompt_luks` helper in `install.sh` only confirms against its own re-entry, never against reality). Less likely: initramfs keyboard layout is wrong, so a symbol you're typing is different from what the kernel receives.

Fix (still have the passphrase in Bitwarden, typo was at install time): you don't — if both copies were typed wrong identically, the LUKS header's slot 0 holds that typo as the real passphrase. Continue reading — there's no magic recovery.

Fix (typo at install, no backup): boot the Ventoy USB → Arch ISO → live env. The Samsung data is unreachable without the passphrase. Full reinstall is the only path — re-run Phase 2 with the corrected passphrase, being careful to type it slowly both times.

Fix (passphrase is correct, layout issue): at the LUKS prompt, try typing slowly. If your passphrase contains symbols (`!@#$%^&*`), the initramfs may be using a US layout even if your physical keyboard is something else. Workaround: reinstall with a passphrase that's ASCII letters + digits only until greetd, then change it later with `sudo cryptsetup luksChangeKey /dev/disk/by-partlabel/ArchRoot`.

Preventive note for re-running Phase 2: `install.sh` enforces an 8+ character minimum and re-prompts on mismatch, but it cannot catch a consistent typo. Type slowly; verify against Bitwarden before hitting Enter the second time.

N-luks. TPM autounlock didn't kick in — still getting the passphrase prompt every boot

Cause: One of three things:

1. `postinstall.sh` skipped the TPM enrollment block (no `/dev/tpm0`, or `SKIP_TPM_LUKS=1` was set).
2. Enrollment succeeded but the PCR values the TPM sealed against have drifted — a firmware update, Secure Boot state change, or bootloader swap between enrollment and now will all do this.
3. The LUKS crypttab entry is missing `tpm2-device=auto` (older `chroot.sh` runs).

Check:

```
# Is there a tpm2 slot?
sudo systemd-cryptenroll /dev/disk/by-partlabel/ArchR
oot
# Expect to see a row with TYPE=tpm2. If only "passwo
rd", enrollment didn't happen.

# Is crypttab.initramfs asking for TPM?
grep tpm2-device /etc/crypttab.initramfs
# Expect: ...luks,discard,tpm2-device=auto
```

Fix (no tpm2 slot): re-run the enrollment manually:

```
sudo systemd-cryptenroll --tpm2-device=auto --tpm2-pc
rs=0+7 /dev/disk/by-partlabel/ArchRoot
# Type the LUKS passphrase when prompted; new slot ad
ded.
sudo reboot
```

Fix (PCR drift): unenroll the stale slot, re-enroll against current PCRs:

```
sudo systemd-cryptenroll --wipe-slot=tpm2 /dev/disk/b
y-partlabel/ArchRoot
sudo systemd-cryptenroll --tpm2-device=auto --tpm2-pc
rs=0+7 /dev/disk/by-partlabel/ArchRoot
sudo reboot
```

Fix (crypttab missing option):

```
sudo sed -i 's/luks,discard$/luks,discard,tpm2-device
=auto/' /etc/crypttab.initramfs
sudo mkinitcpio -P
sudo reboot
```

If TPM enrollment keeps failing entirely, not a blocker — the passphrase path works forever. You type ~6 chars extra per boot; that's it.

K. "Login incorrect" at greetd / first TTY — you fat-fingered `tom`'s password at the install prompt

Cause: `install.sh`'s `prompt_password` accepts any entry that matches its own confirmation. If you typed the same wrong password twice, it hashed that wrong password via `openssl passwd -6` and handed it to `chroot.sh`, which applied it with `chpasswd -e`. No interactive validation ever happened. Same goes for `root`.

Fix — from the Ventoy USB Arch live environment:

```
# Boot Ventoy → Arch ISO → live env.
cryptsetup open /dev/disk/by-partlabel/ArchRoot crypt
root   # LUKS passphrase
mount -o subvol=@ /dev/mapper/cryptroot /mnt
arch-chroot /mnt
passwd tom        # set a new one
passwd            # while you're here, reset root too
if needed
exit
umount -R /mnt
cryptsetup close cryptroot
reboot
```

No other state needs resetting — PAM, keyring, and Bitwarden are all keyed off the new password automatically from the next login.

Recovery

- **Primary recovery**: boot the Ventoy USB, pick the Arch ISO → live environment with `pacstrap`, `arch-chroot`, etc.
- **Secondary recovery**: the Netac has the Arch ISO dd'd onto partition 1 (when using the custom-baked ISO from `pnpm iso`, the dotfiles + scripts are pre-baked into `/root/arch-setup/`). To boot it, use Dell F12 boot menu and pick the Netac's EFI entry — the ISO's own bootloader takes over. So: same live environment without needing the USB.
- **Full reinstall**: boot USB → Arch ISO → re-run `install.sh`. It's size-gated and will abort if anything looks off, so you can't accidentally double-wipe.

Package-name drift (AUR)

If any `yay -S` call in `postinstall.sh` fails with "package not found", the AUR name changed. Search with `yay -Ss <partial>` and substitute. The likely-drifters:

- `bitwarden` (sometimes `bitwarden-desktop`)
- `bitwarden-cli` (stable)
- `pinpam-git` (could be `pinpam` if it ever reaches stable)
- `awww` / `matugen-bin` / `bibata-cursor-translated` / `pacseek` (newer AUR packages — confirm names with `yay -Ss <name>` if any errors)
- `hyprshot` (check `hyprshot-git` from AUR if the stable name disappears)

Re-run `postinstall.sh` after fixing — it's idempotent.